

D. Y. Patil Agriculture and Technical University, Talsande, Kolhapur

Program: BTech – Computer Science and Engineering

Course Name: Computer Organization and Architecture

(Course Code: BTCSE022401)



Course Conducted

By

Mrs. Ashwini N. Sawant

Assistant Professor of CSE,

Department of Computer Science and Engineering,

School of Engineering and Technology,

D. Y. Patil Agriculture and Technical University, Talsande, Kolhapur.

Unit 5:

Control Unit

Control Unit

- The control unit is the part of the computer's central processing unit (CPU) which directs the operation of the processor.
- It fetches instructions from memory, decodes them, and generates control signals to manage the ALU, memory, and I/O devices.

Micro-operations

- Micro-operations are the smallest / elementary operations performed on data inside the CPU.

Role of CU in Micro-operations

Generation of Control Signals

- CU generates control signals required to perform micro-operations .
- These signals activate hardware components like:
 - Registers , ALU , Memory

Example: To perform **Add R1 , R2**

Types of Micro-Operations

1. Register Transfer Micro-operations

Transfer data between registers

Example: $R1 \leftarrow R2$ (Data from R2 is copied to R1)

2. Arithmetic Micro-operations

Perform arithmetic operations

Examples: $R1 \leftarrow R1 + R2$, $R1 \leftarrow R1 - R2$

3. Logic Micro-operations

Perform bitwise operations

Examples: $R1 \leftarrow R1 \text{ AND } R2$, $R1 \leftarrow R1 \text{ OR } R2$, $R1 \leftarrow \text{NOT } R1$

Types of Control Unit

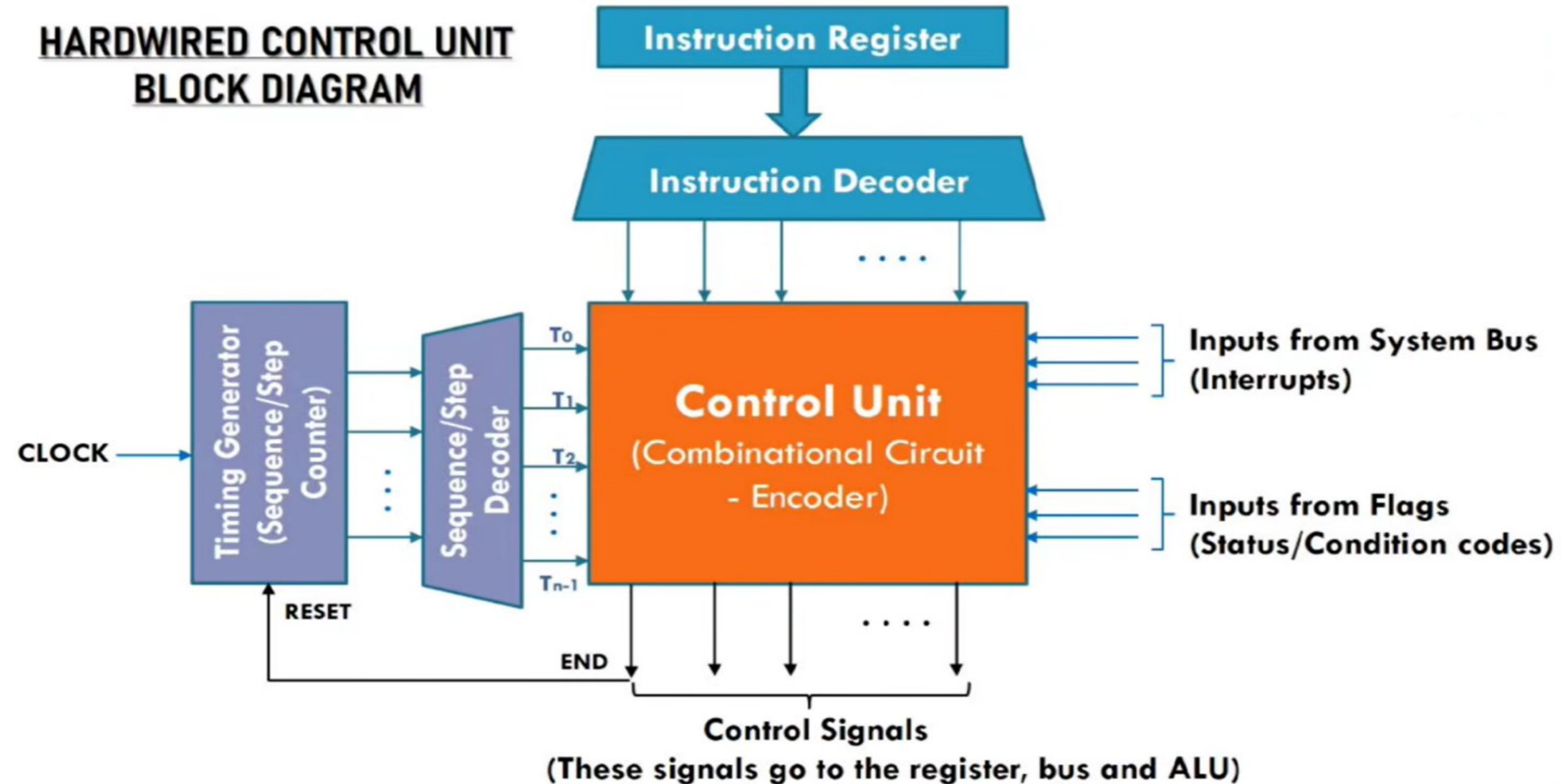
There are two types of control units:

1. Hardwired control unit.
2. Micro programmable control unit.

1) Hardwired Control Unit

- A **Hardwired Control Unit** is a control unit in which control signals are generated using **fixed hardware circuits** such as logic gates, flip-flops, encoders, and decoders.
- It does **not use microprograms**
Control logic is **physically wired**

HARDWIRED CONTROL UNIT BLOCK DIAGRAM



Working of Hardwired Control Unit

Step 1: Instruction Fetch (IR)

- The instruction is fetched from memory.
- It is loaded into the **Instruction Register (IR)**.

This instruction contains: Opcode (operation) , Operand details

Step 2: Instruction Decoding

- The opcode part of the instruction is sent to the **Instruction Decoder**.
- The decoder activates a specific output line corresponding to the instruction.

Example: If instruction = **ADD** → **ADD** line becomes active

- If instruction = **LOAD** → **LOAD** line becomes active

Step 3: Generation of Timing Signals

- . The **Clock** provides continuous pulses.
- . These pulses are given to the **Timing Generator (Sequence Counter)**.

It generates timing steps: **T0, T1, T2, T3, ... Tn**

Step 4: Sequence Step Decoding

- . The timing signals are passed to the **Sequence Step Decoder**.
- . It activates one step at a time.

Step 5: Control Signal Generation (Core Operation)- **Most important step.**

The **Control Unit** receives inputs from:

- Instruction Decoder (type of instruction)
- Timing signals (T0, T1, T2...)

Based on these inputs, the CU generates appropriate **control signals** using logic gates.

Example Logic If:

- Instruction = ADD
- Timing = T3

CU generates: Enable ALU addition ,Load operands , Store result
in form of C0,C1, C2.....,Cn

Step 6: Execution of Instruction

The generated control signals like C0,C1, C5.....,Cn are sent to:

- **Registers** → Load / transfer data
- **ALU** → Perform arithmetic/logic operation
- **Memory** → Read / Write

Operation is executed step-by-step according to timing signals.

Step 7: Handling Interrupts

- Interrupt signals come from the **system bus**.
- CU temporarily stops current execution and services interrupt.

Step 8: End of Instruction

- When instruction execution is complete:
 - **END signal** is generated
 - Sequence counter is **reset**

Step 9: Start Next Instruction Cycle

- Control returns to **T0** , Next instruction is fetched and processed

Overall Flow

**Fetch → Decode → Generate Timing → Generate Control Signals
→ Execute → Reset → Repeat**

Advantages of Hardwired Control Unit

- 1. High Speed:** Control signals are generated directly using hardware
2. Best for Simple Instructions and fixed instruction sets
3. Does not use ROM/RAM for storing control information
- 4. Quick Response Time :** Immediate generation of control signals
5. Suitable for **RISC** Architecture

Disadvantages of Hardwired Control Unit

- 1. Difficult to Modify & complex Design for large instruction sets**
 - Control logic is fixed (wired)
 - Any change requires redesign of hardware
- 2. Not Suitable for CISC**

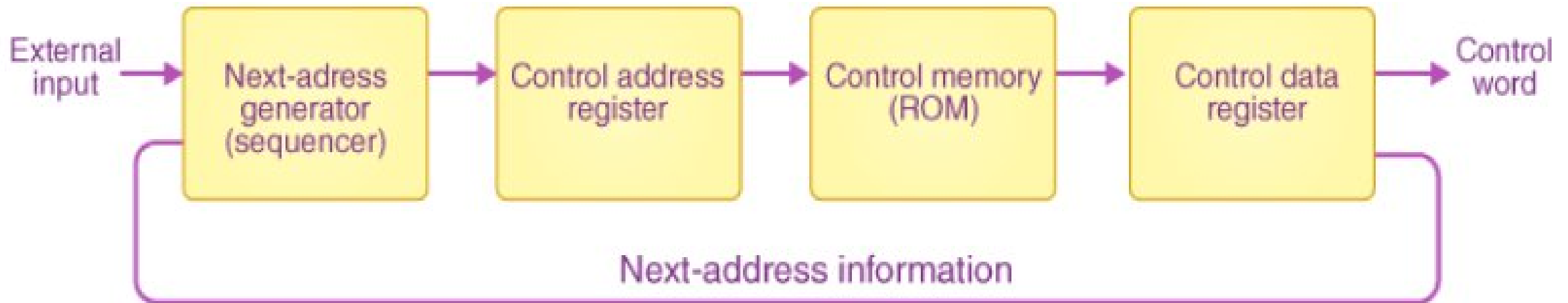
2) Microprogrammed control unit

The programming approach is used to implement a microprogrammed control unit.

The CPU does not follow fixed wiring.

It follows **microinstructions stored in memory** to generate control signals.

(A **microinstruction** is a **control word** stored in control memory)



Microprogrammed control unit of a Basic Computer

Example Instruction : **ADD R1, R2**

(Meaning: $R1 \leftarrow R1 + R2$)

Step 1: Instruction Input

The instruction **ADD R1, R2** is given. Opcode goes to the **Next-Address Generator (Sequencer)**. It decides the **starting address** of the microprogram for **ADD**.

Step 2: Control Address Register (CAR)

Suppose the starting address is **100**.
CAR stores: $CAR \leftarrow 100$

Address	Microinstruction (Control Actions)
100	Load R1 into ALU
101	Add R2 to ALU
102	Store result in R1
103	End / Fetch next instruction

Step 3: All instructions are stored in Control memory ROM .

According to Control Address register (i.e 100) particular microinstruction taken from control memory (ROM) is copied and stored temporarily in CDR (Control Data Register).

Step 4:(Now in Control Data Register CDR)

Microinstruction: $R1 \rightarrow \text{ALU}$

Step 5 :Control Word are generated Control signals :

Enable R1

Enable ALU input

Difference Between Hardwired and Microprogrammed Control Unit

Parameter	Hardwired Control Unit	Microprogrammed Control Unit
Definition	Control signals are generated using fixed hardware circuits (gates, flip-flops, decoders).	Control signals are generated using microinstructions stored in control memory.
Design Approach	Uses combinational and sequential logic circuits.	Uses software-like microprograms.
Speed	Faster (direct hardware execution).	Slower (requires fetching microinstructions).
Flexibility	Less flexible (modification is difficult).	Highly flexible (easy to modify microprogram).
Complexity	Becomes complex for large instruction sets.	Easier to design for complex instruction sets.

Difference Between Hardwired and Microprogrammed Control Unit

Parameter	Hardwired Control Unit	Microprogrammed Control Unit
Control Memory	No control memory required.	Requires control memory (ROM/RAM).
Modification	Difficult (requires hardware changes).	Easy (just update microcode).
Cost	Generally lower for simple CPUs.	Higher due to control memory requirement.
Reliability	More reliable (less dependency on memory).	Slightly less reliable (depends on memory).
Instruction Set Suitability	Suitable for RISC architectures.	Suitable for CISC architectures.
Examples	Used in simple and high-speed systems.	Used in complex processors like older CPUs.

Micro-Instruction Sequencing

Micro-instruction sequencing is the process of determining the address of the next microinstruction to be executed.

Why It Is Needed?

To maintain proper order of execution.

Methods of Sequencing

1. Sequential Sequencing

Next instruction is in sequence

Next Address = Current Address + 1

Example:

100 → 101 → 102 → 103

2. Branching Sequencing

Jump to another address based on condition

Example:

If Z = 1 → Go to 120

Else → Go to 101

3. Mapping (Opcode-based)

Starting address depends on instruction opcode

Example:

ADD → Address 100

SUB → Address 200

Micro-Instruction Execution

Micro-instruction execution is the process of performing operations specified by a microinstruction.

Steps of Execution

1. Fetch Microinstruction

- $CDR \leftarrow M[CAR]$

CDR = *Control Data Register* (also called Control Buffer Register)

CAR = *Control Address Register*

M = *Control Memory*

The contents of the memory location whose address is stored in CAR are transferred to CDR

2. Decode Microinstruction

- Control fields are decoded
- Identify required operations

Micro-Instruction Execution

Micro-instruction execution is the process of performing operations specified by a microinstruction.

Steps of Execution

3. Generate Control Signals

- Signals are sent to:
 - Registers , ALU , Memory

4.Perform Micro-operations

Example:

$$R1 \leftarrow R1 + R2$$

5. Update Next Address

- Sequencer decides next address
- Stored in CAR